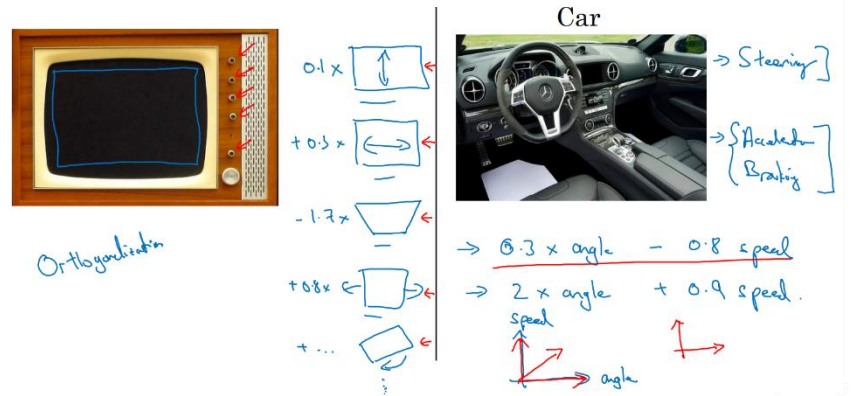


Why Machine Learning Strategy?

- 90% accuracy not good enough
- Ideas: Collect more data, Collect more diverse training set, Train algorithm longer with gradient descent, Try Adam instead of gradient descent, Try bigger network, Try smaller network, Try dropout, Add L2 regularization, Network architecture (activation functions, # hidden units, ...)
- Quick ways to figure out which are worth pursuing and which to safely discard

Orthogonalization

- Effective people are clear-eyed at knowing what to tune to achieve one effect
- Example: TV tuning and car steering



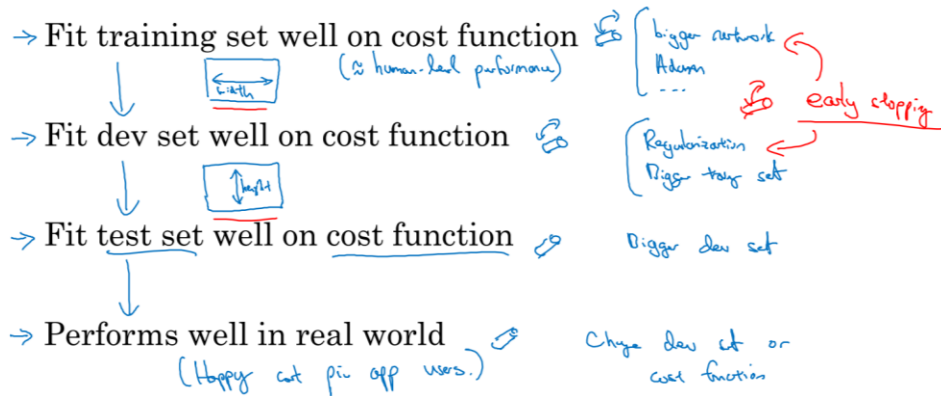
- **Orthogonalization** or orthogonality is a system design property that assures that modifying an instruction or a component of an algorithm will not create or propagate side effects to other components of the system.
- It becomes easier to verify the algorithms independently from one another, it reduces testing and development time.
- 4 assumptions (true and orthogonal) for designing a supervised learning system
 1. **Fit training set well on cost function**

If it doesn't fit well, the use of a bigger neural network or switching to a better optimization algorithm might help.
 2. **Fit development set well on cost function**

If it doesn't fit well, regularization or using bigger training set might help.
 3. **Fit test set well on cost function**

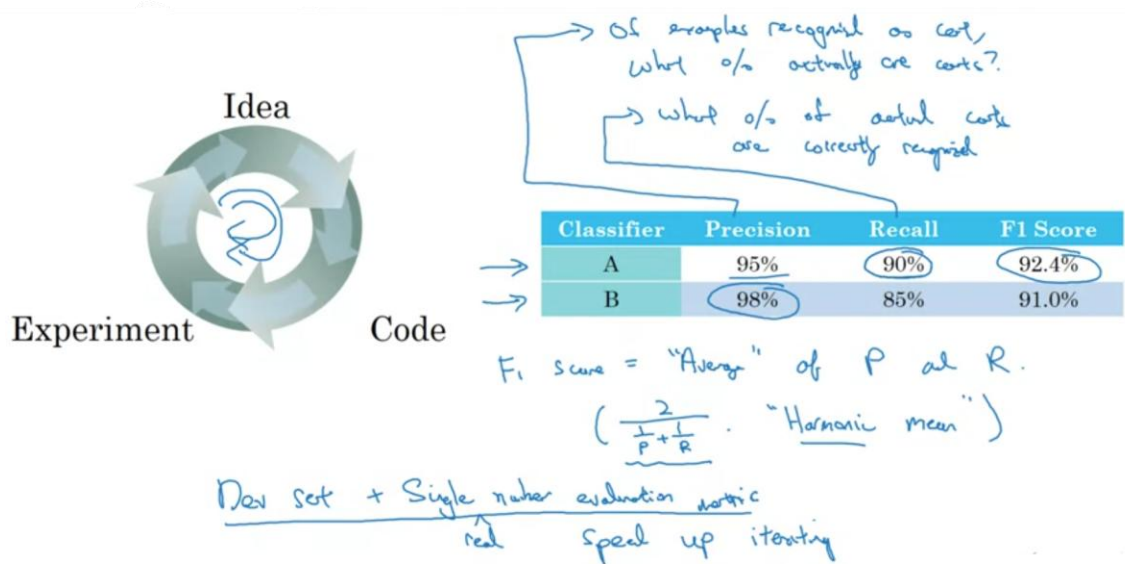
If it doesn't fit well, the use of a bigger development set might help
 4. **Performs well in real world**

If it doesn't perform well, the development test set is not set correctly or the cost function is not evaluating the right thing.
- Early stopping is less orthogonalized because it simultaneously affects training set fit (worse) and dev set fit (better)



Setting Your Goal - Single Number Evaluation Metric

- Quickly tell if the new thing you just tried is better or worse than the last idea
- Empirical process, constant changes during development



- **Precision:** of all examples recognized, what % actually are?

$$\text{Precision (\%)} = \frac{\text{True positive}}{\text{Number of predicted positive}} \times 100 = \frac{\text{True positive}}{(\text{True positive} + \text{False positive})} \times 100$$

- **Recall:** of all images that are, what % were correctly recognized?

$$\text{Recall (\%)} = \frac{\text{True positive}}{\text{Number of predicted actually positive}} \times 100 = \frac{\text{True positive}}{(\text{True positive} + \text{False negative})} \times 100$$

		Actual class y	
		1	0
Predict class $\hat{y}$	1	True positive	False positive
	0	False negative	True negative

- Typically a trade-off

- Combined metric for precision (p) and recall (r) is **F1-score** (harmonic mean)

$$\text{F1-Score} = \frac{2}{\frac{1}{p} + \frac{1}{r}}$$

- Another example:
 - Classifiers achieve direct errors for different users
 - Tracking several numbers makes it hard to quickly pick one
 - Compute a metric like average to quickly see which is best

Algorithm	US	China	India	Other	Average
A	3%	7%	5%	9%	6%
B	5%	6%	5%	10%	6.5%
C	2%	3%	4%	5%	3.5%
D	5%	8%	7%	2%	5.25%
E	4%	5%	2%	4%	3.75%
F	7%	11%	8%	12%	9.5%

Satisficing and Optimizing Metric

- There are different metrics to evaluate classifier performance (**evaluation metrics**)
 - Optimizing** metric: goal to maximize or minimize (accuracy)
 - Satisficing** metric: must meet a min/max acceptable threshold (speed)
- These evaluation matrices must be evaluated on a training set, dev set or test set
- Example:**

Classifier	Accuracy	Running time
A	90%	80ms
B	92%	95ms
C	95%	1,500ms

$\text{Cost} = \text{accuracy} - 0.5 \times \text{Running Time}$

Maximize accuracy
 Subject to Running Time $\leq 100 \text{ ms}$

N metrics: 1 optimizing
 N-1 satisficing

Wakewords / trigger words
 Alexa, OK Google,
 Hey Siri, mihaabaiden
 你好 百度

accuracy.
 #false positive

Maximize accuracy.
 s.t. ≤ 1 false positive
 every 24 hours.

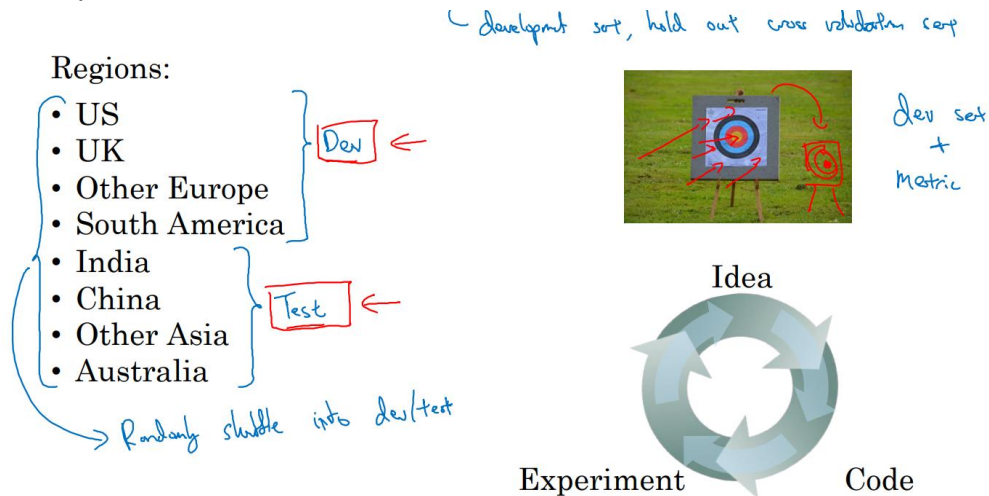
Optimizing metric: accuracy, because you want the classifier to correctly detect a cat image as accurately as possible.

Satisficing metric: running time (100 ms) is the metric the classifier has to meet the expectation of

- General rule: $N_{\text{metric}}: \begin{cases} 1 & \text{Optimizing metric} \\ N_{\text{metric}} - 1 & \text{Satisficing metric} \end{cases}$

Train/Dev/Test Distributions

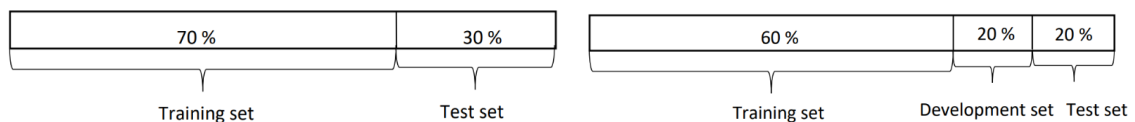
- Setting up the training, development, and test sets have a huge impact on productivity
- It is important to choose the development and test sets from the same distribution and it must be taken randomly from all the data
- Example:



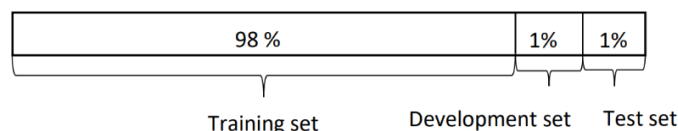
- Example: optimizing on dev set on loan approvals (x → repay loan?) for medium income zip codes, tested on low income zip codes. 3 months wasted
- Guideline: Choose a development set and test set (from the same distribution) to reflect data you expect to get in the future and consider important to do well

Size of Dev and Test Sets

- Old way of splitting data
 - We had smaller set data ($m = 100, 1000, 10000$), therefore we had to use a greater percentage of data to develop and test ideas and models

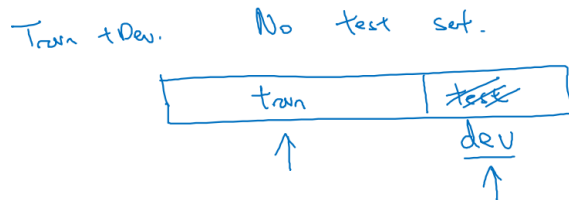


- Modern era – big data
 - Now, because a large amount of data is available ($m = 1,000,000$), we don't have to compromised as much and can use a greater portion to train the model



- Guidelines

- Set up the size of the test set to give a high confidence in the overall performance of the system. Can be okay to not have a test set and just use the dev set if you don't need an unbiased estimate of model performance



- Test set helps evaluate the performance of the final classifier which could be less 30% of the whole data set
- The development set has to be big enough to evaluate different ideas.
- Set your dev set to be big enough to detect differences in algorithm/models you're trying out.

$$\frac{100}{100,000} = 0.1\%$$

$$\frac{10,000}{100,000} = 10\%$$

$$\frac{97\%}{0.1\%} \rightarrow 97.1\%$$

$$\frac{0.01\%}{0.001\%} = 10$$

Only advertising

When to Change Dev/Test Sets and Metrics

- Example: cat classifier

- Original classifier treats each image class equal, but you really want to focus on not misclassifying one subgroup $\rightarrow$ add weight term $w^{(i)} = \{1 \text{ if normal, } 10 \text{ or } 100 \text{ if bad}\}$. Alters normalization constant from $1/m_{\text{dev}}$ to $1/\sum w^{(i)}$

$\rightarrow$ Metric: classification error

Algorithm A: 3% error $\rightarrow$ pornographic

✓ Algorithm B: 5% error

$$\text{Error} = \frac{1}{\sum w^{(i)}} \sum_{i=1}^{m_{\text{dev}}} w^{(i)} \mathbb{I}\{y_{\text{pred}}^{(i)} \neq y^{(i)}\}$$

$$\rightarrow w^{(i)} = \begin{cases} 1 & \text{if } x^{(i)} \text{ is normal} \\ 10 & \text{if } x^{(i)} \text{ is porn} \end{cases}$$

Metric + Dev : Prefer A
 You/users : Prefer B.

- Orthogonalization
 - We've only discussed how to define a metric to evaluate classifiers (place target)
 - Worry separately about how to do well on this metric (aim/shoot accurately)

$$J = \frac{1}{\sum w^{(i)}} \sum_{i=1}^M w^{(i)} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

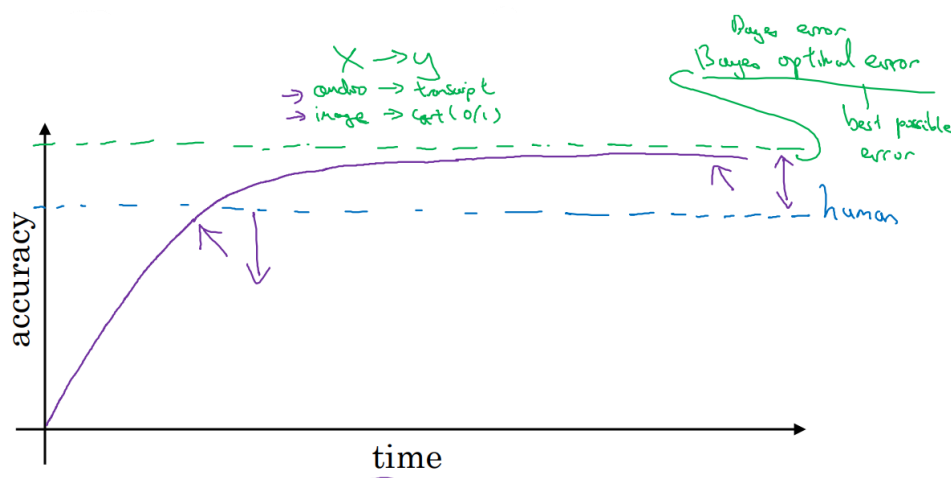
- If doing well on your metric and dev/test set does not correspond to doing well on your application → change your metric and/or dev/test set



- Guideline: define correctly an evaluation metric that helps better rank order classifiers, optimize the evaluation metric

Why Compare to Human-Level Performance?

- Machine learning algorithms can compete with human-level performance since they are more productive and more feasible in a lot of applications
- Workflow of designing and building a machine learning system is much more efficient
- Some of the tasks that humans do are close to “perfection”, which is why machine learning tries to mimic human-level performance

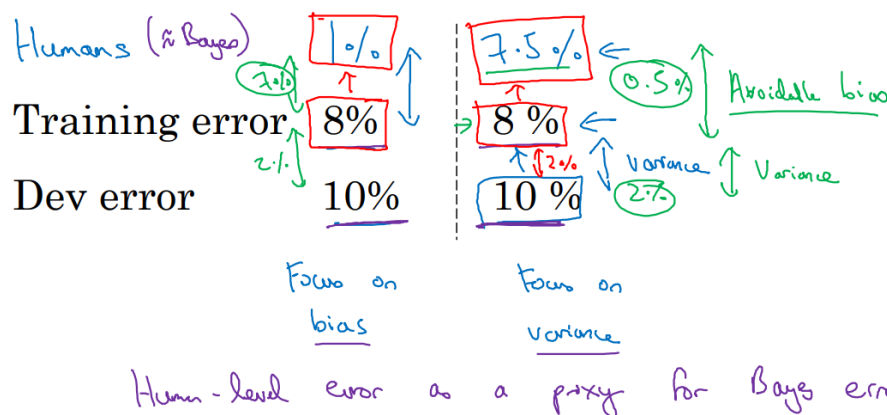


- Progress is rapid as you approach human-level performance
- **Bayes optimal error** is defined as the best possible error

- Any functions mapping from x to y can't surpass a certain level of accuracy
- Machine learning progresses slowly when it surpasses human-level performance
 - Human-level can be close to Bayes optimal error (natural perception)
- When ML performance is worse than the human performance, you can improve it with different tools (harder to use once it surpasses human-level performance)
 - Get labeled data from humans
 - Gain insight from manual error analysis: Why did a person get this right?
 - Better analysis of bias/variance

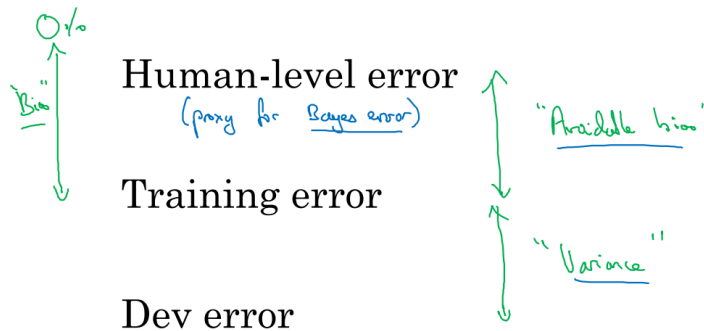
Avoidable Bias

- With human-level performance, can tell when a training set is performing well
- Human level error proxy for Bayes error
- If you want to improve the performance of the training set, you can't do better than the Bayes error → training set is overfitting
- By knowing the Bayes error, it is easier to focus on whether bias or variance avoidance tactics will improve the performance of the model.



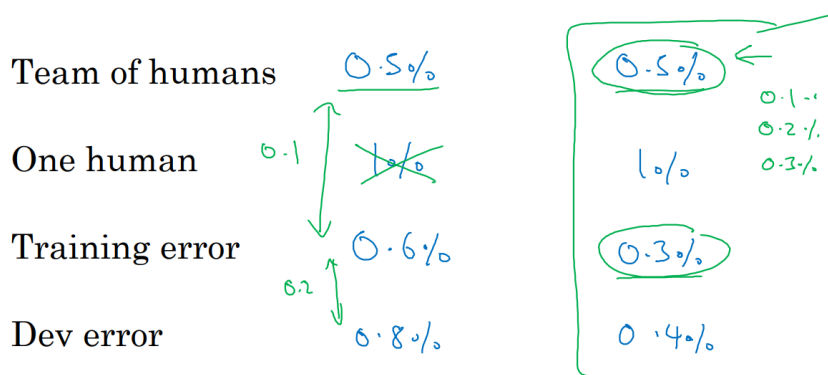
- Scenario A: There is a 7% gap between the performance of the training set and the human level error. It means that the **algorithm isn't fitting well** with the training set since the target is around 1%. To resolve the issue, we use **bias reduction** tactics such as training a bigger neural network or running the training set longer.
- Scenario B: The training set is doing good since there is only a 0.5% difference with the human level error. The difference between the training set and the human level error is called avoidable bias. The focus here is to reduce the variance since the **difference between the training error and the development error** is 2%. To resolve the issue, we use **variance reduction** tactics such as regularization or have a bigger training set.

- If the difference between human-level error and the training error is bigger than the difference between the training error and the development error → **bias reduction**
- If the difference between training error and the development error is bigger than the difference between the human-level error and the training error → **variance reduction**



Surpassing Human-Level Performance

- Example
 - Scenario A: Bayes error is 0.5%, therefore the available bias is 0.1% and the variance is 0.2%.
 - Scenario B: There is not enough information to know if bias reduction or variance reduction should be done on the algorithm. (It doesn't mean that the model cannot be improved, it means that the conventional ways to know if bias reduction or variance reduction are not working in this case.)
Have you overfitted by 0.2%, or is Bayes error actually 0.1%?

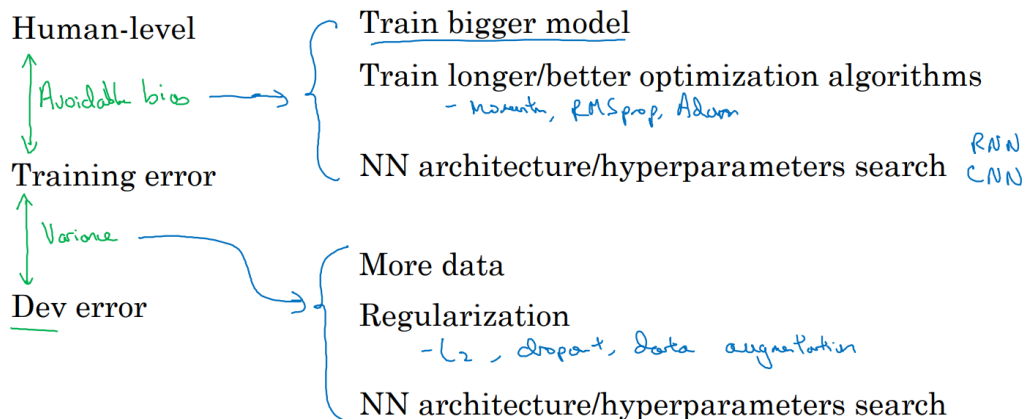


What is avoidable bias?

- There are many problems where machine learning significantly surpasses human-level performance (especially with structured data and lots of it): Online advertising, Product recommendations, Logistics (predicting transit time), Loan approvals, even natural perception problems in a few recent cases

Improving Model Performance

- Two fundamental assumptions of supervised learning:
 - You can fit the training set pretty well (low avoidable bias)
 - Training set performance generalizes pretty well to the dev/test set (low variance)
- $(\text{Human-level error} - \text{training error}) > (\text{training} - \text{dev}) \rightarrow$ **bias reduction**
 - Training a bigger model
 - Train longer/better optimization algorithms (momentum, RMSprop, Adam)
 - NN architecture/hyperparameters search (RNN, CNN)
- $(\text{Training error} - \text{dev error}) > (\text{human-level} - \text{training}) \rightarrow$ **variance reduction**
 - More data set
 - Regularization (L2, dropout, data augmentation)
 - NN architecture/hyperparameters search



Application Scenario: Bird Recognition in the City of Peacetopia

This scenario is adapted from a real-world application, with specifics altered for confidentiality.



As a distinguished researcher in the City of Peacetopia, you face a unique challenge. The citizens are universally afraid of birds, and your task is to develop an algorithm to detect birds flying over the city and alert the populace.

The City Council has provided you with 10,000,000 sky images from Peacetopia's security cameras, labeled as follows:

- $y = 0$: There is no bird on the image
- $y = 1$: There is a bird on the image

Your objective is to engineer an algorithm that can accurately classify new security camera images. Decisions regarding the evaluation metric and data structuring into train/dev/test sets are critical.

The City Council has outlined their desires for the algorithm:

1. High accuracy.
2. Quick response time for classifying new images.
3. Low memory requirement to function on small processors across various cameras.

True or False: You realize that having a single evaluation metric will expedite development and ease algorithm comparison.

False

True

After further discussions, the city narrows down its criteria to:

- "We need an algorithm that can let us know a bird is flying over Peacetopia as accurately as possible."
- "We want the trained model to take no more than 10 seconds to classify a new image."
- "We want the model to fit in 10MB of memory."
- "We require a minimum of 98% test accuracy."

If you had the three following models, which one would you choose?

Test Accuracy	Runtime	Memory size
97%	1 sec	3MB
Test Accuracy	Runtime	Memory size
99%	13 sec	9MB
Test Accuracy	Runtime	Memory size
98%	9 sec	9MB
Test Accuracy	Runtime	Memory size
97%	3 sec	2MB

Not trying to optimize runtime or memory size, just stay under the threshold (satisficing metrics). Optimize accuracy given the thresholds are met.

Based on the context of a city's data analysis project, which of the following statements is true regarding the metrics used?

Accuracy, running time, and memory size are all optimizing metrics because you want to do well on all three.

Accuracy, running time, and memory size are all satisficing metrics because you have to do sufficiently well on all three for your system to be acceptable.

Accuracy is an optimizing metric; running time and memory size are satisficing metrics.

Accuracy is a satisficing metric; running time and memory size are an optimizing metric.

Before implementing your algorithm, you need to split your data into train/dev/test sets. You have a dataset of 10,000,000 examples. Which of these do you think is the best choice?

Train	Dev	Test
3,333,334	3,333,333	3,333,333
Train	Dev	Test
6,000,000	1,000,000	3,000,000
Train	Dev	Test
6,000,000	3,000,000	1,000,000
Train	Dev	Test
9,500,000	250,000	250,000

You propose a 95% / 2.5% / 2.5% for train / dev / test splits to the City Council. They ask for your reasoning. Which of the following best justifies your proposal, given that the total data set contains 10,000,000 data points?

The emphasis on the training set provides the most accurate model, supporting the memory and processing efficiency.

With a dataset comprising 10,000,000 individual samples, 2.5% represents 250,000 samples, which should be more than enough for dev and testing to evaluate bias and variance.

The most important goal is achieving the highest accuracy, and that can be done by allocating the maximum amount of data to the training set.

The emphasis on the training set will allow us to iterate faster.

Now that you've set up your train/dev/test sets, the City Council comes across another 1,000,000 images from social media and offers them to you. These images have a different distribution from the images the City Council originally provided, but you think they could help your algorithm. Which of the following is the best use of that additional data?

Do not use the data. It will change the distribution of any set it is added to.

Add it to the dev set to evaluate how well the model generalizes across a broader set.

Split it among train/dev/test equally.

Add it to the training set.

Now that you've set up your train/dev/test sets, the City Council comes across another 1,000,000 images from social media and offers them to you. These images have a different distribution from the images the City Council originally provided, but you think they could help your algorithm. Should you add this data to the training set?

No. **Yes.**

One member of the City Council wants to add 1,000,000 citizen data images to the development (dev) set. Your original data is from security cameras, and you object to adding the citizen data because: (Choose all that apply)

The dev set no longer reflects the distribution of data (security cameras) you most care about.

A bigger test set will slow down the speed of iterating because of the computational expense of evaluating models on the test set.

The 1,000,000 citizen data images do not have a consistent input-output relationship as the security camera data.

This would cause the dev and test set distributions to become different. This is a bad idea because you're not aiming where you want to hit.

One member of the City Council wants to add 1,000,000 citizen data images evenly to the training, development (dev), and test sets. Your original data is from security cameras, and you object because:

The additional data would significantly slow down training time.

The training set will not be as accurate because of the different distributions.

If we add the images to the test set, then it won't reflect the distribution of data (security cameras) expected in production.

The 1,000,000 citizen data images do not have a consistent input-output relationship as the security camera data.

You train a system, and its errors are as follows (error = 100% - Accuracy%):

Training set error	4.0%
Dev set error	4.5%

This suggests that one good avenue for improving performance is to train a bigger network to reduce the 4.0% training error. Do you agree?

No, because there is insufficient information to tell.

Yes, because having a 4.0% training error shows you have a high bias.

Yes, because this shows your bias is higher than your variance.

No, because this shows your variance is higher than your bias.

Human performance for identifying birds is < 1%, training set error is 5.2%, and dev set error is 7.3%. Which of the options below is the best next step?

Train a bigger network to reduce the 5.2% training error.

Validate the human data set with a sample of your data to ensure the images are of sufficient quality.

Try an ensemble model to reduce bias and variance.

Get more data or apply regularization to reduce variance.

You want to define "human-level performance" for a bird species identification project to present to the city council. Which of the following is the best way to define it?

The average performance of all the city's ornithologists (0.5%).

The average of all recorded error rates (0.66%, ornithologists and citizens).

The performance of the city's best ornithologist (0.3% error rate).

The average performance of regular citizens of Peacetopia (1.2%).

Which of the following statements do you agree with?

A learning algorithm's performance can be better than human-level performance and better than Bayes error.

A learning algorithm's performance can never be better than human-level performance nor better than Bayes error.

A learning algorithm's performance can be better than human-level performance but it can never be better than Bayes error.

A learning algorithm's performance can never be better than human-level performance but it can be better than Bayes error.

Which of the below shows the optimal order of accuracy from worst to best?

Human-level performance -> the learning algorithm's performance -> Bayes error.

The learning algorithm's performance -> Bayes error -> human-level performance.

The learning algorithm's performance -> human-level performance -> Bayes error.

Human-level performance -> Bayes error -> the learning algorithm's performance.

A team of experts achieves 0.1% error on a complex image classification task, establishing this as the human-level performance. Your algorithm's performance is as follows:

Human-level performance	0.1%
Training set error	2.0%
Dev set error	2.1%

Based on this, which two options are the most promising to pursue?

Increase the training dataset size to reduce variance.

Increase regularization to reduce variance.

Decrease regularization to allow the model to better fit the training data and reduce bias.

Increase model complexity to improve performance on the training set, addressing the high bias.

You've now also run your model on the test set and find that the error rate is 7.0% compared to a 2.1% error rate for the dev set. What should you do? (Choose all that apply)

Try increasing regularization to reduce overfitting to the dev set.

Get a bigger test set to increase its accuracy.

Try decreasing regularization for better generalization with the dev set.

Increase the size of the dev set.

You also evaluate your model on the test set and find the following:

Human-level performance	0.1%
Training set error	2.0%
Dev set error	2.1%
Test set error	7.0%

What does this mean? (Check the two best options.)

You should try to get a bigger dev set.

You have underfitted to the dev set.

You have overfit to the dev set.

You should get a bigger test set.

After working on this project for a year, you finally achieve: Human-level performance, 0.10%, Training set error, 0.05%, Dev set error, 0.05%. Which of the following are likely? (Check all that apply.)

There is still avoidable bias.

This result is not possible since it should not be possible to surpass human-level performance.

Pushing to even higher accuracy will be slow because you will not be able to easily identify sources of bias.

The model has recognized complex, emergent features that humans may not readily perceive. (Chess and Go, for example).

After working on this project for a year, you finally achieve: Human-level performance, 0.10%, Training set error, 0.05%, Dev set error, 0.05%. Which of the following are true? (Check all that apply.)

All or almost all of the avoidable bias has been accounted for.

With only 0.05% further progress to make, you should quickly be able to close the remaining gap to 0%.

You are close to Bayes error and there is a possibility of overfitting.

It is highly unlikely this result is purely a statistical anomaly, but statistical noise may still contribute to the error.

It turns out Peacetopia has hired one of your competitors to build a system as well. Your system and your competitor both deliver systems with about the same running time and memory size. However, your system has higher accuracy!

Still, when Peacetopia tries out both your system and your competitor's system, they conclude they actually like your competitor's system better, because even though you have higher overall accuracy, you have more false negatives (failing to raise an alarm when a bird is in the air).

What should you do?

Pick false negative rate as the new metric, and use this new metric to drive all further development.

Ask your team to take into account both accuracy and false negative rate during development.

Rethink the appropriate metric for this task, and ask your team to tune to the new metric.

Look at all the models you've developed during the development process and find the one with the lowest false negative error rate.

You've handily beaten your competitor, and your system is now deployed in Peacetopia and is protecting the citizens from birds! But over the last few months, a new species of bird has been slowly migrating into the area, so the performance of your model is being tested on a new type of data.

There are only 1,000 images of the new species. The city expects a better system from you within the next 3 months.

Which of these should you do first?

Add the new images and split them among train/dev/test.

Augment your data to increase the number of images of the new bird species.

Add hidden layers to further refine feature development.

Put the 1,000 images into the dev set to evaluate the bias and re-tune.

The City Council thinks that having more cats in the city would help scare off birds. They are so happy with your work on the Bird detector that they also hire you to build a Cat detector. You have a huge dataset of 100,000,000 cat images. Training on this data takes about two weeks. Which of the statements do you agree with? (Check all that agree.)

Given a significant budget for cloud GPUs, you could mitigate the training time.

Accuracy should exceed the City Council's requirements, but the project may take as long as the bird detector because of the two-week training/iteration time.

With the experience gained from the Bird detector, you are confident to build a good Cat detector on the first try.

You could consider a tradeoff where you use a subset of the cat data to find reasonable performance with reasonable iteration pacing.

Because of years of working on Cat detectors, you have such a huge dataset of 100,000,000 cat images that training on this data takes about two weeks.

Which of the statements do you agree with? (Choose all that apply.)

If 100,000,000 examples is enough to build a good enough Cat detector, you might be better off training with just 10,000,000 examples to gain a ~10x improvement in how quickly you can run experiments, even if each model performs a bit worse because it's trained on less data.

Buying faster computers could speed up your team's iteration speed and thus your team's productivity.

Needing two weeks to train will limit the speed at which you can iterate.

Having built a good Bird detector, you should be able to take the same model and hyperparameters and just apply it to the Cat dataset, so there is no need to iterate.

Carrying Out Error Analysis

- Manually examining mistake the algorithm is making for insights about what to do next
- Look at dev examples to evaluate ideas
 - Ceiling:** upper bound on performance improvement
 - Simple counting can help a lot with direction in a short amount of time



90% accuracy
→ 10% error

Should you try to make your cat classifier do better on dogs? ←

Error analysis:

- Get ~100 mislabeled dev set examples.
- Count up how many are dogs.

→ 5%
5/100

10%
9.5%

"ceiling"

→ 50%
50/100

10%
↓
5%

- Evaluate multiple ideas in parallel
 - Create a percent table (spreadsheet) of images to manually analyze and error category/reason for misrecognition
 - Estimate of how worthwhile each category might be to work on
- Fix pictures of dogs being recognized as cats ←
- Fix great cats (lions, panthers, etc..) being misrecognized ←
- Improve performance on blurry images ←

Image	Dog	Great Cats	Blurry	Instagram	Comments
1	✓			✓	Pitbull
2			✓	✓	
3		✓	✓		Rainy day at zoo
⋮	⋮	⋮	⋮		
% of total	8%	43%	61%	12%	

Cleaning Up Incorrectly Labeled Data

- Training set
 - DL algorithms are quite robust to random errors in the training set
 - As long as errors are not too far from **random**, then it's probably okay to just leave them as they are
 - Less robust to **systematic** errors



- Dev/test set errors
 - Add column for incorrect labels
 - If it makes a significant difference to your ability to evaluate algorithms on your dev set, spend the time to fix incorrect labels
 - Overall dev set error = incorrect label errors + errors from other causes

	Image	Dog	Great Cat	Blurry	Incorrectly labeled	Comments
	...					
	98				✓	Labeler missed cat in background
	99		✓			
	100				✓	Drawing of a cat; Not a real cat.
	% of total	8%	43%	61%	6%	

Overall dev set error 10%
 Errors due incorrect labels 0.6% ←
 Errors due to other causes 9.4% ←

2%
 0.6%
 1.4%
 2.1% 1.9%

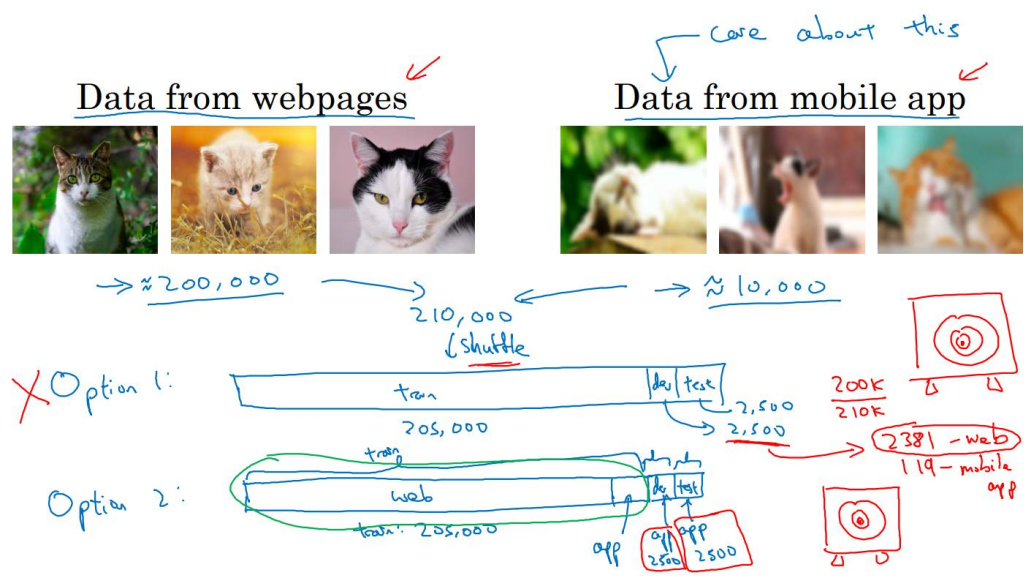
- Goal of dev set is to help you select between two classifiers A and B. If you don't trust your dev set to correctly tell you whether this classifier is better than the other, then there's a good reason to go in and fix labels in dev set
- Correcting incorrect dev/test set examples
 - Apply same process to your dev and test sets to make sure they continue to come from the same distribution
 - Consider examining examples your algorithm got right and ones it got wrong
 - Train and dev/test data may now come from slightly different distributions

Build Your First System Quickly, Then Iterate

- Example: speech recognition
 - Noisy background, accented speech, far from microphone, stuttering, ...
- Guideline: Build your first system quickly, then iterate
 - **Set up development/ test set and metrics**
 - Set up a target
 - **Build an initial system quickly**
 - Train training set quickly: Fit the parameters
 - Development set: Tune the parameters
 - Test set: Assess the performance
 - **Use bias/variance analysis & error analysis to prioritize next steps**

Training and Testing on Different Distributions

- Guideline: choose a development set and test set to reflect data you expect to get in the future and consider important to do well on
- Cat app example
 - Option 1: Combine and randomly shuffle: same distribution, but very few of the smaller group in the dev/test sets
 - ***Option 2**: Dev and test sets all training examples from the group you care about



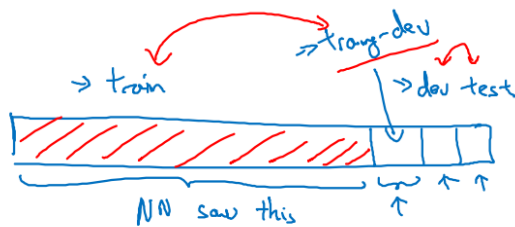
Advantage: target is well defined

Disadvantage: training dist is different from the dev and test set dist

However, this way of splitting the data has **better performance in long term**

Bias And Variance with Mismatched Data Distributions

- When the training set is from a different distribution than the development and test sets, the method to analyze bias and variance changes
- Algorithm sees data in training set but not dev set. What if dev set is just harder to classify? Also the distribution is different. How do you know what causes the difference in training error and dev error?
- **Training-dev set:** new subset of data that should have the same distribution as the training set, but isn't used to train the NN



	Classification error (%)					
	Scenario A	Scenario B	Scenario C	Scenario D	Scenario E	Scenario F
Human (proxy for Bayes error)	0	0	0	0	0	4
Training error	1	1	1	10	10	7
Training-development error	-	9	1.5	11	11	10
Development error	10	10	10	12	20	6
Test error	-	-	-	-	-	6

Scenario A: Since the training data and the dev data come from a different distribution, cannot conclude large variance problem (algorithm not generalizing well from training set). The problem might be that the dev set is more difficult to classify accurately

Scenario B: The error between the training set and the training-dev set is 8%. Since the training set and training-dev set come from the same distribution, the only difference between them is the neural network sorted the data in the training and not in the training development. The neural network is **not generalizing well to data from the same distribution that it hadn't seen before**. Therefore, we have really a **variance problem**

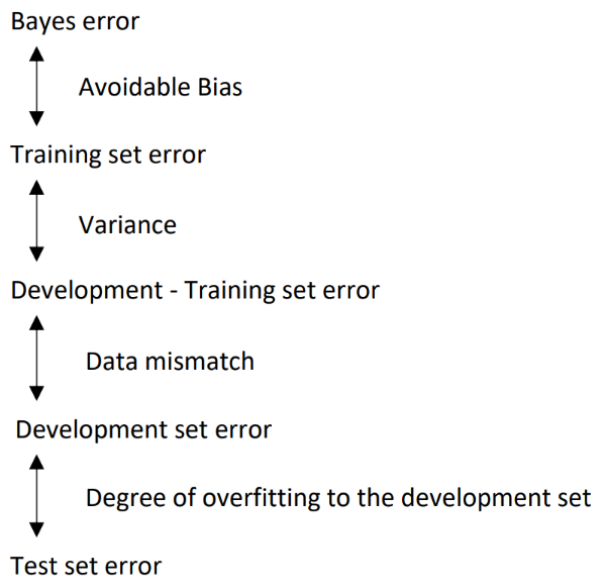
Scenario C: In this case, we have a **mismatch data problem** since the 2 data sets **come from different distributions**

Scenario D: In this case, the **avoidable bias** is high since **the difference between Bayes error and training error** is 10%

Scenario E: In this case, there are 2 problems. The first one is that the **avoidable bias** is high since the difference between Bayes error and training error is 10 % and the second one is a **data mismatched** problem

Scenario F: Development should never be done on the test set. However, the difference between the development set and the test set gives the degree of overfitting to the development set

- General formulation

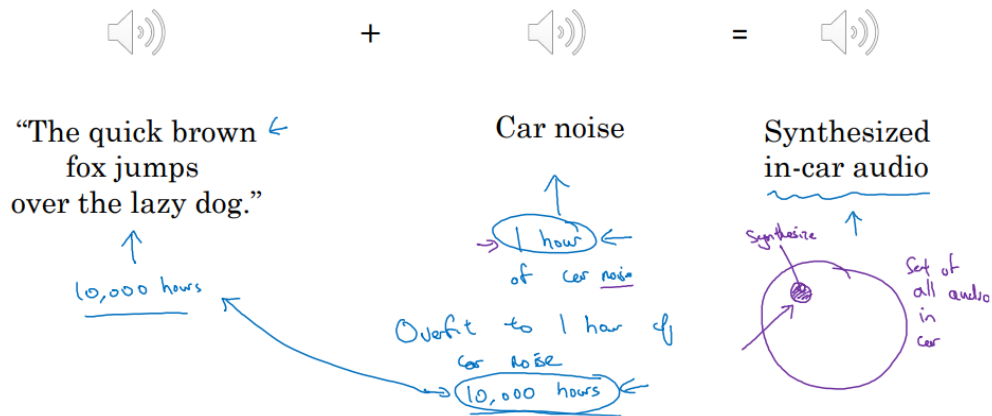


	Grand speech recognition	Recurse mirror speech data.	
Human level	"Human level" 4%	6%	↕ avoidable bias
Error on examples trained on	"Training error" 7%	6%	
Error on examples <u>not</u> trained on	"Training-dev error" 10%	"Dev/Test error" 6%	↕ Variance

↔ data mismatch

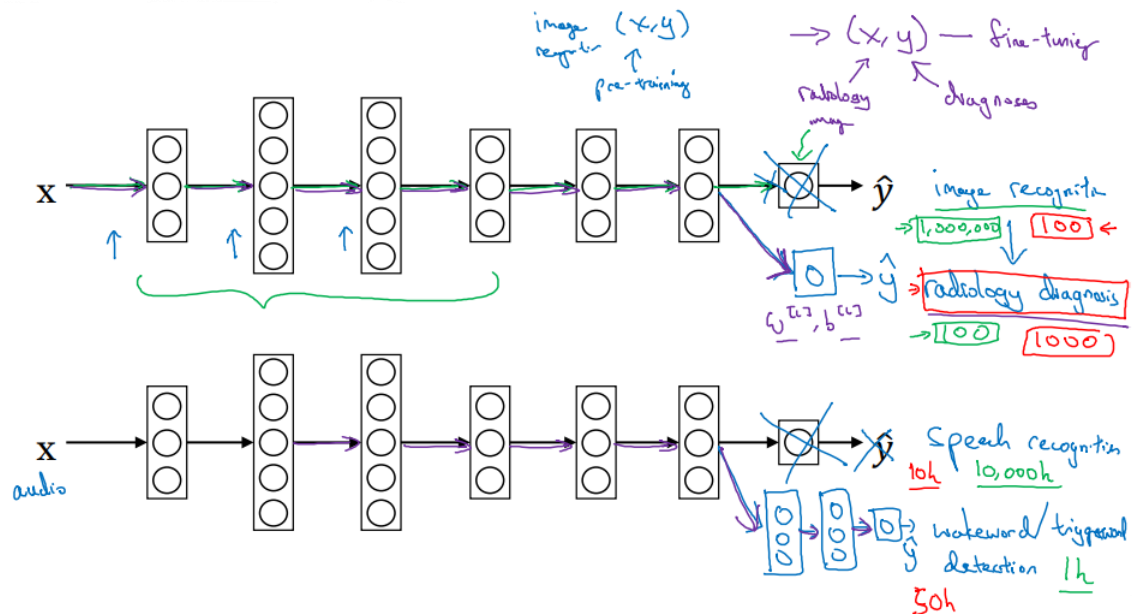
Addressing Data Mismatch

- Carry out manual error analysis to understand the error differences between training and dev/test sets.
 - Development should never be done on test set to avoid overfitting
- Make training data more similar or collect more data similar to dev and test sets
 - To make the training data more similar to your development set, you can use artificial data synthesis. However, it is possible that **you might be accidentally simulating data only from a tiny subset of the space of all possible examples**
- **Artificial data synthesis**



Learning from Multiple Tasks – Transfer Learning

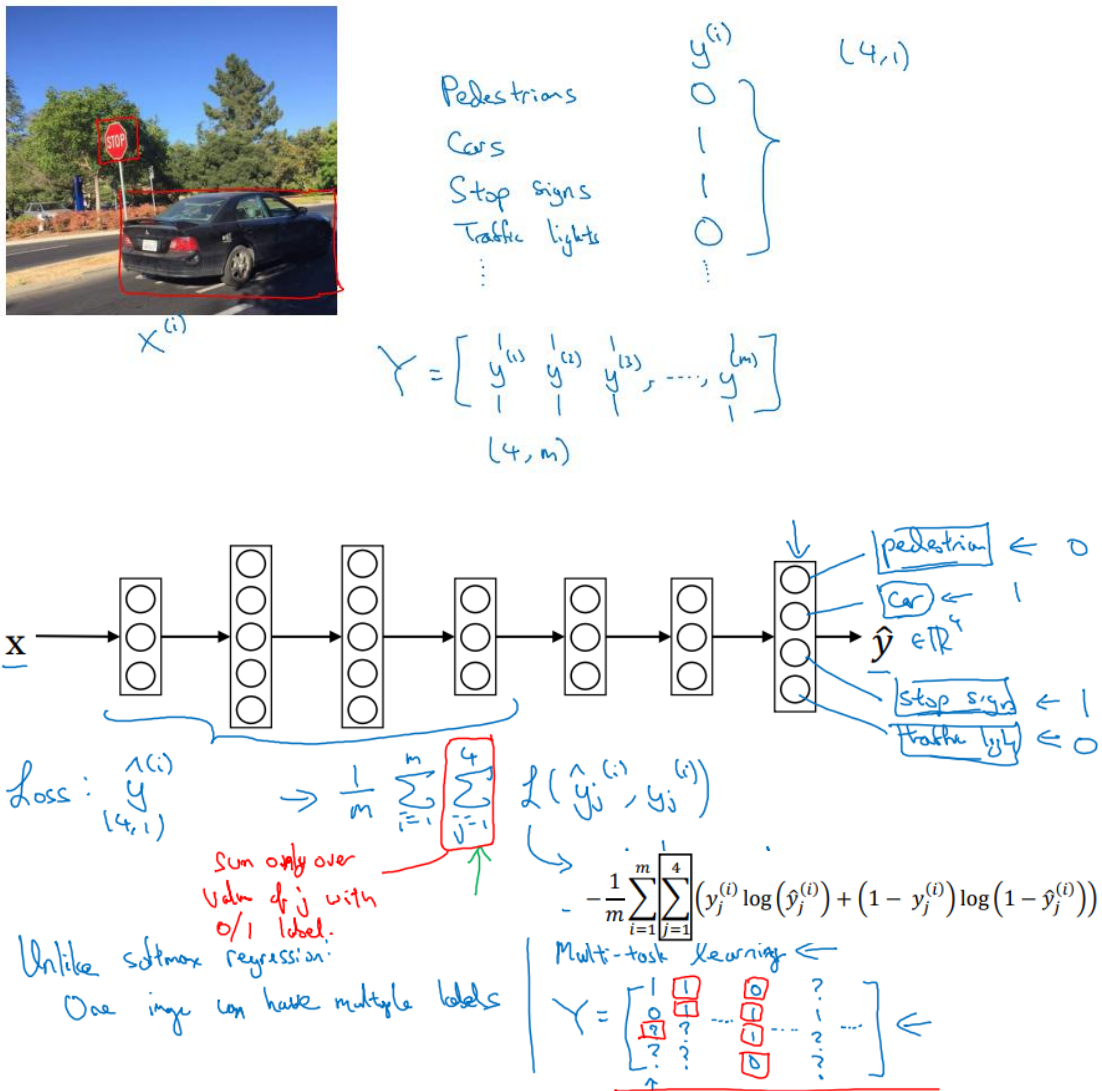
- Take knowledge NN has learned from one task and apply to a separate task



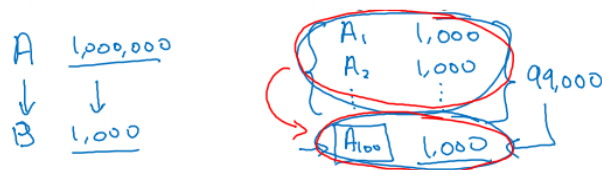
- Pre-training:** initial phase of training where the NN will learn about the structure and the nature of images (pre-initializes the weight)
- Fine-tuning:** updating all the weights afterwards
- Guideline:**
 - Remove output layer of neural network (can add more layers before new output)
 - Delete weights feeding into the output layer
 - Create a new set of randomly initialized weights for the last layer only
 - Feed new data set (x, y) to whole NN
- When transfer learning (from A $\rightarrow$ B) makes sense
 - Task A and B have the same input x
 - You have a lot more data for Task A (large m) than Task B (small m)
 - Low level features from A could be helpful for learning B

Multi-Task Learning

- Example: simplified autonomous driving



- When multi-task learning makes sense
 - Training on tasks that could benefit from having shared lower-level features
 - Usually: amount of data you have for each task is similar

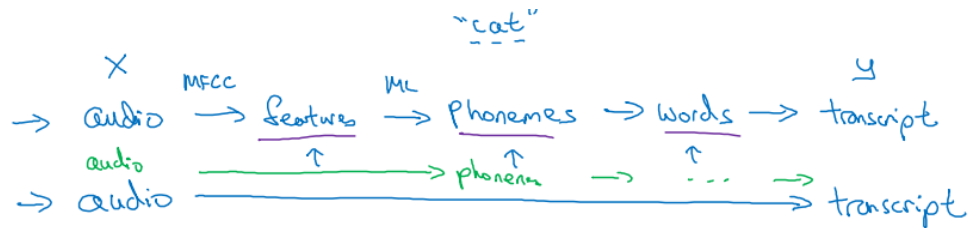


99,000 extra training examples

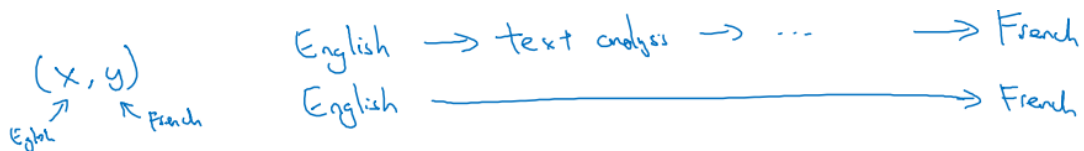
- Can train a big enough neural network to do well on all the tasks
- Only time multi-task learning hurts is when the NN isn't big enough

End-to-End Deep Learning

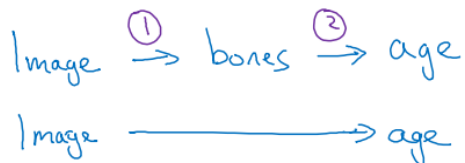
- Simplification of several stages of processing into one neural network
- You need a LOT of data for this. It may be easier to get data for sub-tasks
- Example: Speech recognition
 - MFCC: algorithm for extracting a certain set of hand designed features for audio
 - You should only use full end-to-end when you have a large dataset. Traditional pipeline works better for small datasets
 - Some researchers utilize the intermediate steps they developed pre-end-to-end



- Face recognition
 - Two steps: locate face/zoom in, and identify
 - Why this works better: each problem is simpler, and there is enough data for each problem (not enough for full problem at once)
- Machine translation



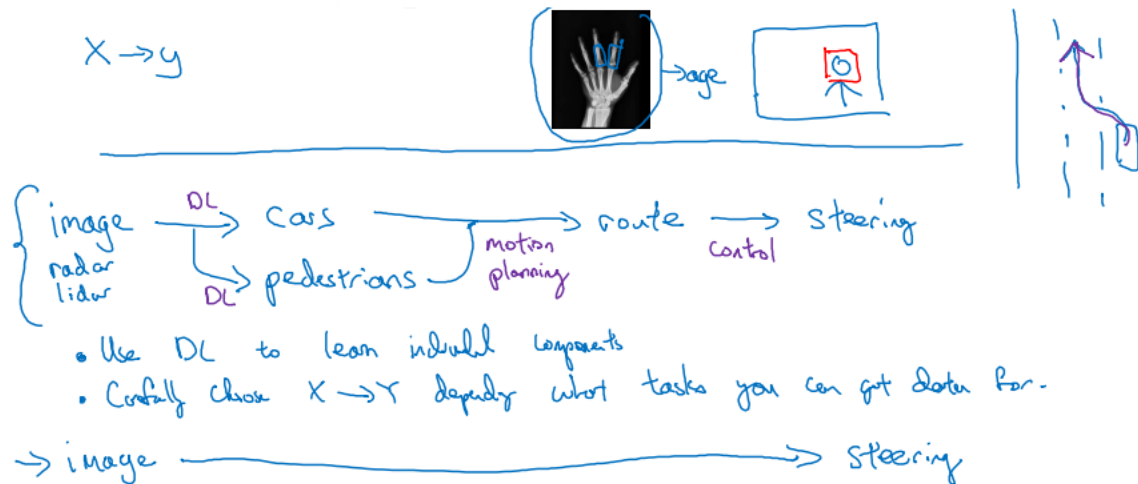
- Child age estimation from x-ray



- Cannot be used for every problem since it needs a lot of labeled data

Whether to use End-to-End Learning

- Pros:
 - Lets the data speak (NN will find the right function, may be more able to capture whatever statistics are in the data, rather than being forced to reflect human preconceptions)
 - Less hand-designing of components needed (simplifies design workflow)
- Cons:
 - May need large amount of data
 - Excludes potentially useful hand-designed components (inject manual knowledge into algorithm; especially helpful when you don't have much data)
- Do you have enough data to learn a function of the complexity needed to map x and y ?
 - If not, use DL to learn individual components
 - Carefully choose what types of X to Y mapping you want to learn, depending on what tasks you can get data for
 -



To help you practice strategies for machine learning, this week we'll present another scenario and ask how you would act. We think this “simulator” of working in a machine learning project will give you an idea of what leading a machine learning project could be like!

You are employed by a startup building self-driving cars. You are in charge of detecting road signs (stop sign, pedestrian crossing sign, construction ahead sign) and traffic signals (red and green lights) in images. The goal is to recognize which of these objects appear in each image. As an example, the image below contains a pedestrian crossing sign and red traffic lights.



$$y^{(i)} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} \begin{matrix} \text{"stop sign"} \\ \text{"pedestrian crossing sign"} \\ \text{"construction ahead sign"} \\ \text{"red traffic light"} \\ \text{"green traffic light"} \end{matrix}$$

Your 100,000 labeled images are taken using the front-facing camera of your car. This is also the distribution of data you care most about doing well on. You think you might be able to get a much larger dataset off the internet, which could be helpful for training even if the distribution of internet data is not the same.

You are getting started with this project. What is the first thing you do? *Assume each of the steps below would take about an equal amount of time (a few days).*

Spend a few days checking how well humans perform at these tasks.

Spend a few days training a basic model and see what mistakes it makes.

Spend a few days getting the internet data, so that you understand better what data is available.

Spend a few days collecting more data using the front-facing camera of your car, to better understand how much data per unit time you can collect.

Your goal is to detect road signs (stop sign, pedestrian crossing sign, construction ahead sign) and traffic signals (red and green lights) in images. The goal is to recognize which of these objects appear in each image. You plan to use a deep neural network with ReLU units in the hidden layers. Suppose that you use a sigmoid function for the output layer, and the output $\hat{y}$ has shape (5, 1). Which of the following best describes the cost function?

$$\frac{1}{m} \sum_{i=1}^m (-y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$$

$$\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^5 \mathcal{L}(\hat{y}_i^{(j)}, y_i^{(j)})$$

$$\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^5 \mathcal{L}(\hat{y}_j^{(i)}, y_j^i)$$

$$\frac{\exp \hat{y}_j^{(i)}}{\sum_{j=1}^5 \exp \hat{y}_j^{(i)}}$$

You are working out error analysis and counting up what errors the algorithm makes. Which of the following do you think you should manually go through and carefully examine, one image at a time?

500 images of the test set, on which the algorithm made a mistake.

500 images of the train set, on which the algorithm made a mistake.

500 images of the dev set, on which the algorithm made a mistake.

500 images of the training-dev set, on which the algorithm made a mistake.

After working on the data for several weeks, your team ends up with the following data:

- 100,000 labeled images taken using the front-facing camera of your car.
- 900,000 labeled images of roads downloaded from the internet.
- Each image's labels precisely indicate the presence of any specific road signs and

traffic signals or combinations of them. For example, $y^{(i)} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ means the image contains a stop sign and a red traffic light.

When using a non-fully labeled image such as $y^{(i)} = \begin{bmatrix} 0 \\ ? \\ 1 \\ ? \\ 1 \end{bmatrix}$, which of the following strategies is most appropriate to calculate the loss function to train as a multi-task learning problem?

Calculate the loss as $\sum \mathcal{L}(\hat{y}_j^{(i)}, y_j^{(i)})$ where the sum goes over all the known components of $y^{(i)}$.

It is not possible to use non-fully labeled images if we train as a multi-task learning problem.

Make the missing entries equal to 1.

Make the missing entries equal to 0.

True or False: The distribution of data you care about contains images from your car's front-facing camera, which comes from a different distribution than the images you were able to find and download off the internet. The best way to split the data is using the 900,000 internet images to train, and divide the 100,000 images from your car's front-facing camera between dev and test sets.

True

False

(100,000 images are too many to use in dev and test. A better distribution would be to use 80,000 of those images to train, and split the rest between dev and test. This ensures that your dev and test sets reflect the "real-world" data you care about.)

Assume you've finally chosen the following split between the data:

Dataset:	Contains:	Error of the algorithm:
Training	940,000 images randomly picked from (900,000 internet images + 60,000 car's front-facing camera images)	1%
Training-Dev	20,000 images randomly picked from (900,000 internet images + 60,000 car's front-facing camera images)	5.1%
Dev	20,000 images from your car's front-facing camera	5.6%
Test	20,000 images from the car's front-facing camera	6.8%

You also know that human-level error on the road sign and traffic signals classification task is around 0.5%. Which of the following is true?

The size of the train-dev set is too large.

You have a high bias.

You have a high variance problem.

You have a large data-mismatch problem.

Assume you've finally chosen the following split between the data:

Dataset:	Contains:	Error of the algorithm:
Training	940,000 images randomly picked from (900,000 internet images + 60,000 car's front-facing camera images)	2%
Training-Dev	20,000 images randomly picked from (900,000 internet images + 60,000 car's front-facing camera images)	2.3%
Dev	20,000 images from your car's front-facing camera	1.3%
Test	20,000 images from the car's front-facing camera	1.1%

Human-level error on this task is approximately 0.5%. (Bayes error is the lowest possible error rate for a task. Human-level error is a good estimation of Bayes error.)

True or False: Based on this, the Bayes error for the car camera images (Dev/Test) is higher than the Bayes error for the mixed internet/car images (Training).

False True

The Dev/Test errors (1.3% and 1.1%) are lower than the Training/Training-Dev errors (2% and 2.3%). This indicates the car camera images are likely inherently easier for the algorithm to learn, thus having a lower Bayes error.

You decide to focus on the dev set and check by hand what the errors are due to. Here is a table summarizing your discoveries:

Overall dev set error	15.3%
Errors due to incorrectly labeled data	4.1%
Errors due to foggy pictures	8.0%
Errors due to rain drops stuck on your car's front-facing camera	2.2%
Errors due to other causes	1.0%

In this table, 4.1%, 8.0%, etc. are a fraction of the total dev set (not just examples of your algorithm mislabeled). For example, about $8.0/15.3 = 52\%$ of your errors are due to foggy pictures.

True/False: Should the team's highest priority be to bring more foggy pictures into the training set to address the 8.0% of errors in that category?

True because it is greater than the other error categories added together ($8.0 > 4.1 + 2.2 + 1.0$).

True because it is the largest category of errors. We should always prioritize the largest category of errors as this will make the best use of the team's time.

False because it depends on how easy it is to add foggy data. If foggy data is very hard and costly to collect, it might not be worth the team's effort.

First start with the sources of error that are least costly to fix.

You decide to focus on the dev set and check by hand what the errors are due to. Here is a table summarizing your discoveries:

Overall dev set error	15.3%
Errors due to incorrectly labeled data	4.1%
Errors due to foggy pictures	3.0%
Errors due to partially occluded elements.	7.2%
Errors due to other causes	1.0%

In this table, 4.1%, 7.2%, etc. are a fraction of the total dev set (not just examples of your algorithm mislabeled). For example, about $7.2/15.3 = 47\%$ of your errors are due to partially occluded elements in the image.

True or False: From this table, you can conclude that if you fix the incorrectly labeled data you will reduce the overall dev set error to 11.2%.

True

False

The categories aren't mutually exclusive. The 4.1% only gives you an estimate of the ceiling of how much the error can be improved by fixing the labels.

You decide to use data augmentation to address foggy images. You find 1,000 pictures of fog off the internet, and “add” them to clean images to synthesize foggy days, like this:



Which one of the following statements do you agree with?

Adding synthesized images that look like real foggy pictures taken from the front-facing camera of your car to the training dataset won't help the model improve because it will introduce avoidable bias.

As long as the synthesized fog looks realistic to the human eye, you can be confident that the synthesized data is accurately capturing the distribution of real foggy images (or a subset of it), since human vision is very accurate for the problem you're solving.

There is little risk of overfitting to the 1,000 pictures of fog as long as you are combining it with a much larger (much greater than 1,000) set of clean/non-foggy images.

After working further on the problem, you've decided to correct the incorrectly labeled data on the dev set. Which of these statements do you agree with?

You should not correct the incorrectly labeled data in the test set, so that the dev and test sets continue to come from the same distribution.

You do not necessarily need to fix the incorrectly labeled data in the training set because it's acceptable for the training set distribution to differ from the dev and test sets. Note that it is important that the dev set and test set have the same distribution.

You should correct incorrectly labeled data in the training set as well to avoid your training set now being even more different from your dev set.

You should also correct the incorrectly labeled data in the test set, so that the dev and test sets continue to come from the same distribution.

One of your colleagues at the startup is starting a project to classify road signs as stop, dangerous curve, construction ahead, dead-end, and speed limit signs. Given how specific the signs are, he has only a small dataset and hasn't been able to create a good model. You offer your help providing the trained weights (parameters) of your model to transfer knowledge.

True or False: Your colleague points out that his problem has more specific items than the ones you used to train your model. This makes the transfer of knowledge impossible.

False True

The model can benefit from the pre-trained model since there are many features learned by your model that can be used in the new problem.

Another colleague wants to use microphones placed outside the car to better hear if there are other vehicles around you. For example, if there is a police vehicle behind you, you would be able to hear their siren. However, they don't have much data to train this audio system. How can you help?

Neither transfer learning nor multi-task learning seems promising.

Transfer learning from your vision dataset could help your colleague get going faster. Multi-task learning seems significantly less promising.

Either transfer learning or multi-task learning could help our colleague get going faster.

Multi-task learning from your vision dataset could help your colleague get going faster. Transfer learning seems significantly less promising.

The problem involves different data types and tasks, making it unlikely for either transfer learning or multi-task learning to be effective.

When building a system to detect cattle crossing a road from images taken with the front-facing camera of a truck, the designers have a large dataset of images. Which of the following is the strongest reason to use an end-to-end approach?

It is the default approach for all computer vision tasks.

A large dataset is available.

It requires fewer computational resources.

It relies on manually designed components.

Consider the following two approaches, A and B:

- (A) Input an image (x) to a neural network and have it directly learn a mapping to make a prediction as to whether there's a red light and/or green light (y).
- (B) In this two-step approach, you would first (i) detect the traffic light in the image (if any), then (ii) determine the color of the illuminated lamp in the traffic light.

Approach A tends to be more promising than approach B if you have a _____ (fill in the blank).

Large training set.

Large bias problem.

Multi-task learning problem.